SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

KUNDURU SRINIVASA REDDY
192525452D

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

- Q1 Master Theorem: Four Subproblems — 4 marks
- Q2 Master Theorem: Unequal Growth — 4 marks
- Q3 Master Theorem: Square-Root Work — 4 marks
- Q4 Master Theorem: Five Subproblems — 4 marks

Q1 Master Theorem: Four Subproblems — 4 marks

Analyze $T(n) = 4T(n/2) + n^2 \log n$ using the Master Theorem. Identify a, b, and f(n), state whether a standard case applies, and justify the asymptotic bound.

Your Answer

Given recurrence: $T(n) = 4T(n/2) + n^2 \log n$

Identifying parameters:

* a = 4 (number of subproblems)
* b = 2 (factor by which problem size is divided)

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

KUNDURU SRINIVASA REDDY
192525452D

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

- Q1 Master Theorem: Four Subproblems — 4 marks
- Q2 Master Theorem: Unequal Growth — 4 marks
- Q3 Master Theorem: Square-Root Work — 4 marks
- Q4 Master Theorem: Five Subproblems — 4 marks

Q2 Master Theorem: Unequal Growth — 4 marks

Analyze $T(n) = 3T(n/2) + n$ using the Master Theorem, where applicable. Explain which term dominates.

Your Answer

The recurrence $T(n) = 3T(n/2) + n$ can be analyzed using the Master Theorem, which applies to recurrences of the form $T(n) = aT(n/b) + f(n)$.

Here, a = 3, b = 2, and f(n) = n.

Step 1: Compute n^(log_b a)

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

KUNDURU SRINIVASA REDDY
192525452D

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

- Q1 Master Theorem: Four Subproblems — 4 marks
- Q2 Master Theorem: Unequal Growth — 4 marks
- Q3 Master Theorem: Square-Root Work — 4 marks
- Q4 Master Theorem: Five Subproblems — 4 marks

Q3 Master Theorem: Square-Root Work — 4 marks

Find the asymptotic complexity of T(n) = 2T(n/4) + √n. State the parameters and justify the applicable case.

Your Answer

Given recurrence: T(n) = 2T(n/4) + √n

Master Theorem Parameters:

* a = 2 (number of subproblems)
* b = 4 (factor by which problem size shrinks)

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

KUNDURU SRINIVASA REDDY
192525452D

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

- Q1 Master Theorem: Four Subproblems — 4 marks
- Q2 Master Theorem: Unequal Growth — 4 marks
- Q3 Master Theorem: Square-Root Work — 4 marks
- Q4 Master Theorem: Five Subproblems — 4 marks
- Q5

Q4 Master Theorem: Five Subproblems — 4 marks

A divide-and-conquer algorithm makes 5 recursive calls on subproblems of size n/3 and performs $\Theta(n^2)$ extra work. Determine its asymptotic time complexity.

Your Answer

Given recurrence: T(n) = 5T(n/3) + $\Theta(n^2)$

Master Theorem parameters:

* a = 5 (number of subproblems)
* b = 3 (factor by which subproblem size shrinks)

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

KUNDURU SRINIVASA REDDY
192525452D

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

- Q1 Master Theorem: Four Subproblems — 4 marks
- Q2 Master Theorem: Unequal Growth — 4 marks
- Q3 Master Theorem: Square-Root Work — 4 marks
- Q4 Master Theorem: Five Subproblems — 4 marks

Q5 Master Theorem: Logarithmic Factor — 4 marks

Analyze T(n) = 16T(n/4) + n^2/log n. Discuss whether the standard Master Theorem applies directly and provide a justified bound.

Your Answer

T(n) = 16T(n/4) + n^2/log n

Step 1: Identify Master Theorem parameters

Here, a = 16, b = 4, f(n) = n^2/log n.

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

KUNDURU SRINIVASA REDDY
192525452D

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

Q1 Master Theorem: Four Subproblems — 4 marks
Q2 Master Theorem: Unequal Growth — 4 marks
Q3 Master Theorem: Square-Root Work — 4 marks
Q4 Master Theorem: Five Subproblems — 4 marks

Q6 Substitution: Linearithmic Recurrence — 4 marks

Use the substitution method to prove an asymptotic bound for T(n) = 2T(n/2) + n. State an induction hypothesis and base case.

Your Answer

Claim: T(n)= Θ(n log n). We will prove T(n)≤ c · n log n for the upper bound O(n log n).

1. Induction Hypothesis:
Assume that for all k < n$, $T(k) \leq c \cdot k \log_2 k$ for some constant $c > 0 ParseError: Can't use function '$' in math mode at position 6: k < n$_ $T(k) \leq c ....

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

KUNDURU SRINIVASA REDDY
192525452D

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

Q1 Master Theorem: Four Subproblems — 4 marks
Q2 Master Theorem: Unequal Growth — 4 marks
Q3 Master Theorem: Square-Root Work — 4 marks
Q4 Master Theorem: Five Subproblems — 4 marks

Q7 Substitution: Decreasing by One — 4 marks

Use substitution or expansion to establish a tight asymptotic bound for T(n) = T(n−1) + n, assuming T(1)=1.

Your Answer

To find the tight asymptotic bound for T(n)= T(n − 1)+ n with T(1)= 1, we can use the expansion method:

1. Expand the recurrence:

T(n)= T(n − 1)+ n
T(n − 1)= T(n − 2)+ (n − 1)
T(n − 2)= T(n − 3)+ (n − 2)

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

KUNDURU SRINIVASA REDDY
192525452D

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

- Q1 Master Theorem: Four Subproblems — 4 marks
- Q2 Master Theorem: Unequal Growth — 4 marks
- Q3 Master Theorem: Square-Root Work — 4 marks
- Q4 Master Theorem: Five Subproblems — 4 marks

Q8 Substitution: Halving with Constant Work — 4 marks

Prove that T(n) = T(n/2) + 1 is Θ(log n), assuming n is a power of two.

Your Answer

To prove T(n)= T(n/2)+ 1 is Θ(log n), we will use the substitution method (mathematical induction).

1. Base Case:
Assume n = 2^0 = 1.
T(1)= T(1/2)+ 1. In recurrence relations, we typically define a base case such as T(1)= c.
For log_2 (1)= 0, the bound holds.

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

KUNDURU SRINIVASA REDDY
192525452D

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

- Q1 Master Theorem: Four Subproblems — 4 marks
- Q2 Master Theorem: Unequal Growth — 4 marks
- Q3 Master Theorem: Square-Root Work — 4 marks
- Q4 Master Theorem: Five Subproblems — 4 marks

Q9 Substitution: Exponential Recurrence — 4 marks

Establish a tight asymptotic bound for T(n) = 2T(n−1) + 1 with T(0)=1. Explain the growth pattern.

Your Answer

To establish a tight asymptotic bound for the recurrence T(n)= 2T(n − 1)+ 1 with T(0)= 1, we can use the method of substitution (unrolling):

1. Unrolling the recurrence:

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

KUNDURU SRINIVASA REDDY
192525452D

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

- Q1 Master Theorem: Four Subproblems — 4 marks
- Q2 Master Theorem: Unequal Growth — 4 marks
- Q3 Master Theorem: Square-Root Work — 4 marks
- Q4 Master Theorem: Five Subproblems — 4 marks

Q10 Check a Complexity Claim — 4 marks

An analyst claims T(n) = T(n/2) + n is Θ(n log n). Assess the claim and establish the correct asymptotic bound.

Your Answer

The claim that T(n)= T(n/2)+ n is Θ(n log n) is incorrect.

To find the correct bound, we apply the Master Theorem for recurrences of the form T(n)= aT(n/b)+ f(n):

1. Identify parameters: a = 1, b = 2, and f(n)= n.
2. Calculate n^(log_b a): n^(log_2 1) = n^0 = 1.

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

KUNDURU SRINIVASA REDDY
192525452D

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

- Q1 Master Theorem: Four Subproblems — 4 marks
- Q2 Master Theorem: Unequal Growth — 4 marks
- Q3 Master Theorem: Square-Root Work — 4 marks
- Q4 Master Theorem: Five Subproblems — 4 marks

Q11 Algorithm Crossover — 4 marks

Algorithm A performs $5n \log_2 n$ operations and Algorithm B performs $0.02n^2$ operations. Compare them and estimate the crossover input size.

Your Answer

To find the crossover point, we set the two complexity functions equal to each other:
$5n \log_2(n) = 0.02n^2$

Dividing both sides by n:
$5 \log_2(n) = 0.02n$

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

KUNDURU SRINIVASA REDDY
192525452D

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

- Q1 Master Theorem: Four Subproblems — 4 marks
- Q2 Master Theorem: Unequal Growth — 4 marks
- Q3 Master Theorem: Square-Root Work — 4 marks
- Q4 Master Theorem: Five Subproblems — 4 marks

Q12 Scalability: n log n — 4 marks

A program processes 10,000 records in 2 seconds using an $O(n \log_2 n)$ algorithm. Estimate runtime for 100,000 records, assuming proportional scaling and negligible fixed overhead.

Your Answer

To estimate the runtime, we use the ratio of the complexities:

Ratio = (n_2 · log_2 (n_2))/(n_1 · log_2 (n_1))
Ratio = (100,000 · log_2 (100,000))/(10,000 · log_2 (10,000))
Ratio = (100,000 · 16.61)/(10,000 · 13.29)
Ratio ≈ 10 · 1.25 = 12.5

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

KUNDURU SRINIVASA REDDY
192525452D

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

- Q1 Master Theorem: Four Subproblems — 4 marks
- Q2 Master Theorem: Unequal Growth — 4 marks
- Q3 Master Theorem: Square-Root Work — 4 marks
- Q4 Master Theorem: Five Subproblems — 4 marks

Q13 Scalability: Quadratic — 4 marks

An $O(n^2)$ algorithm takes 4 seconds for 2,000 items. Estimate runtime for 6,000 items, assuming the quadratic term dominates.

Your Answer

36 seconds

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

KUNDURU SRINIVASA REDDY
192525452D

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

- **Q1** Master Theorem: Four Subproblems — 4 marks
- **Q2** Master Theorem: Unequal Growth — 4 marks
- **Q3** Master Theorem: Square-Root Work — 4 marks
- **Q4** Master Theorem: Five Subproblems — 4 marks

Q14 Compare Three Growth Rates — 4 marks

Compare O(n), O(n log n), and O(n^2) growth. Discuss how their operation counts scale and give an appropriate use case for each.

Your Answer

The growth rates of O(n), O(n log n), and O(n^2) represent different tiers of algorithmic efficiency as the input size n increases:

1. O(n) - Linear Growth:
   * Scaling: The operation count increases in direct proportion to the input size. If the input doubles, the time taken roughly doubles. This is highly efficient for large datasets.
   * Use Case: Searching for an element in an unsorted array using Linear Search or traversing a linked list.

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

KUNDURU SRINIVASA REDDY
192525452D

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

- Q1 Master Theorem: Four Subproblems — 4 marks
- Q2 Master Theorem: Unequal Growth — 4 marks
- Q3 Master Theorem: Square-Root Work — 4 marks
- Q4 Master Theorem: Five Subproblems — 4 marks

Q15 Constant Factors and Big-O — 4 marks

Two algorithms take $T_1(n)=100n$ and $T_2(n)=n \log_2 n$. Explain why asymptotic notation alone may not determine which is faster for small inputs.

Your Answer

Asymptotic notation (like Big-O) describes the growth rate of a function as n approaches infinity, effectively ignoring constant factors and lower-order terms. While $T_2(n) = n \log_2 n$ has a better asymptotic complexity than $T_1(n) = 100n$, the large constant factor of 100 in $T_1(n)$ dominates when n is small. For example, if $n = 2$, $T_1(n) = 200$ while $T_2(n) = 2$. In this small-scale range, $T_2(n)$ is significantly faster. The crossover point where the logarithmic growth of $T_2(n)$ eventually overcomes the large constant of $T_1(n)$ may only occur at a much higher value of n, meaning asymptotic analysis alone is insufficient to predict performance for small input sizes.

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

KUNDURU SRINIVASA REDDY
192525452D

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

- Q1 Master Theorem: Four Subproblems — 4 marks
- Q2 Master Theorem: Unequal Growth — 4 marks
- Q3 Master Theorem: Square-Root Work — 4 marks
- Q4 Master Theorem: Five Subproblems — 4 marks

Q16 Hash Set: Time and Space — 4 marks

An algorithm scans an array once and stores distinct values in a hash set. Analyze expected time and auxiliary space, stating assumptions about hash operations.

Your Answer

To analyze the complexity of an algorithm that scans an array once and stores distinct values in a hash set, we make the following assumptions:

1. The hash set operations (insert and lookup) take O(1) average time complexity, assuming a good hash function and a reasonable load factor.
2. The input array has a size of n.

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

KUNDURU SRINIVASA REDDY
192525452D

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

- Q1 Master Theorem: Four Subproblems — 4 marks
- Q2 Master Theorem: Unequal Growth — 4 marks
- Q3 Master Theorem: Square-Root Work — 4 marks
- Q4 Master Theorem: Five Subproblems — 4 marks

Q17 Recursive Temporary Arrays — 4 marks

A recursive algorithm splits an array into two halves, processes both halves, and allocates a temporary array of size n at each call. Analyze time and peak auxiliary space, clarifying whether allocations overlap.

Your Answer

Time Complexity Analysis:
The recurrence relation for this algorithm can be expressed as:
T(n)= 2T(n/2)+ O(n)
Where:

* 2T(n/2) represents the two recursive calls on half-sized arrays.

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

KUNDURU SRINIVASA REDDY
192525452D

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

- Q1 Master Theorem: Four Subproblems — 4 marks
- Q2 Master Theorem: Unequal Growth — 4 marks
- Q3 Master Theorem: Square-Root Work — 4 marks
- Q4 Master Theorem: Five Subproblems — 4 marks

Q18 Nested Loops with a Condition — 4 marks

An algorithm has three loops from 1 to n, but the innermost statement executes only when i+j≤n. Determine its time complexity and justify the count.

Your Answer

The time complexity of the algorithm is Θ(n^2).

Justification:

1. Loop Structure: The algorithm consists of three nested loops. Let's denote the loop variables as i, j, and k.
2. The Condition: The innermost statement only executes when i + j ≤ n. This condition depends only on the first two loops (i

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

KUNDURU SRINIVASA REDDY
192525452D

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

- Q1 Master Theorem: Four Subproblems — 4 marks
- Q2 Master Theorem: Unequal Growth — 4 marks
- Q3 Master Theorem: Square-Root Work — 4 marks
- Q4 Master Theorem: Five Subproblems — 4 marks

Q19 Matrix Allocation vs Traversal — 4 marks

A program allocates an n×n matrix but traverses only the main diagonal. Distinguish total space complexity from traversal time complexity.

Your Answer

In this scenario, there is a significant distinction between space complexity and time complexity:

1. Space Complexity: Even though the algorithm only visits the main diagonal, the act of allocating an n × n matrix requires memory proportional to the total number of elements. Therefore, the space complexity is O(n^2).

2. Time Complexity: Since the program only traverses the main diagonal, it only visits n elements (where each element is at

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

KUNDURU SRINIVASA REDDY
192525452D

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

- Q1 Master Theorem: Four Subproblems — 4 marks
- Q2 Master Theorem: Unequal Growth — 4 marks
- Q3 Master Theorem: Square-Root Work — 4 marks
- Q4 Master Theorem: Five Subproblems — 4 marks

Q20 Binary Search: Time and Space — 4 marks

Compare iterative and recursive binary search in time and auxiliary space. Explain the role of recursion-stack overhead.

Your Answer

In terms of time complexity, both the iterative and recursive versions of Binary Search have a complexity of O(log n), as both methods repeatedly halve the search space.

However, they differ significantly in auxiliary space complexity:

1. Iterative Binary Search: This approach uses a simple loop and a few variables (like low, high, and mid) to track the search

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

KUNDURU SRINIVASA REDDY
192525452D

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

- Q1 Master Theorem: Four Subproblems — 4 marks
- Q2 Master Theorem: Unequal Growth — 4 marks
- Q3 Master Theorem: Square-Root Work — 4 marks
- Q4 Master Theorem: Five Subproblems — 4 marks

Q21 Unequal Subproblem Sizes — 4 marks

Analyze T(n)=T(n/3)+T(2n/3)+n using a recursion-tree argument. Explain the work per level and total tree depth.

Your Answer

To analyze the recurrence T(n)= T(n/3)+ T(2n/3)+ n using a recursion tree, we look at the work done at each level of the tree:

1. Work per Level:

* At the root (Level 0), the work done is n.

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

KUNDURU SRINIVASA REDDY
192525452D

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

- Q1 Master Theorem: Four Subproblems — 4 marks
- Q2 Master Theorem: Unequal Growth — 4 marks
- Q3 Master Theorem: Square-Root Work — 4 marks
- Q4 Master Theorem: Five Subproblems — 4 marks

Q22 Unequal Split with Linear Combine — 4 marks

A divide-and-conquer algorithm splits into subproblems of sizes n/4 and 3n/4 and performs Θ(n) combining work. Determine its asymptotic time complexity using a suitable method.

Your Answer

To solve the recurrence T(n)= T(n/4)+ T(3n/4)+ Θ(n), we use the Recursion Tree method.

1. Work per level: At the root, the work is n. At the first level of recursion, the work is (n/4)+(3n/4)= n. At the second level, the work is (n/16 + 3n/16)+(3n/16 + 9n/16)= n. In general, the total work at each level of the tree is n.
2. Tree Depth: The recurrence branches unevenly. The longest path to a leaf follows the 3n/4 branch, giving a depth of log_(4/3) n. The shortest path follows the n/4 branch, giving a depth of log_4 n.

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

KUNDURU SRINIVASA REDDY
192525452D

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

- Q1 Master Theorem: Four Subproblems 4 marks
- Q2 Master Theorem: Unequal Growth 4 marks
- Q3 Master Theorem: Square-Root Work 4 marks
- Q4 Master Theorem: Five Subproblems 4 marks

Q23 Fibonacci-Style Recurrence — 4 marks

A recursive procedure satisfies $T(n)=T(n-1)+T(n-2)+\Theta(1)$. Determine its asymptotic growth and explain the source of the growth.

Your Answer

The recurrence relation $T(n)= T(n - 1)+ T(n - 2)+ \Theta(1)$ describes a process where each step branches into two smaller subproblems of sizes $n - 1$ and $n - 2$, with a constant amount of work $\Theta(1)$ performed at each level.

Asymptotic Growth:
The asymptotic growth of this recurrence is exponential, specifically $T(n)= \Theta(\phi^n)$, where ϕ is the Golden Ratio ≈ 1.618.

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

KUNDURU SRINIVASA REDDY
192525452D

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

- Q1 Master Theorem: Four Subproblems 4 marks
- Q2 Master Theorem: Unequal Growth 4 marks
- Q3 Master Theorem: Square-Root Work 4 marks
- Q4 Master Theorem: Five Subproblems 4 marks

Q24 Four Half-Size Subproblems — 4 marks

Design and analyze a recurrence for an algorithm that splits into four subproblems of size n/2 and spends $\Theta(n)$ combining their results.

Your Answer

The recurrence relation for the described algorithm is:
T(n) = 4T(n/2) + \Theta(n)

To analyze this using the Master Theorem:

* a = 4

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

KUNDURU SRINIVASA REDDY
192525452D

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

- Q1 Master Theorem: Four Subproblems 4 marks
- Q2 Master Theorem: Unequal Growth 4 marks
- Q3 Master Theorem: Square-Root Work 4 marks
- Q4 Master Theorem: Five Subproblems 4 marks
- Q5

Q25 Time-Space Trade-off — 4 marks

One algorithm uses O(n) time and $O(n^2)$ space; another uses O(n log n) time and O(n) space. Discuss which is more appropriate in a memory-constrained environment.

Your Answer

In a memory-constrained environment, the second algorithm (O(n log n) time and O(n) space) is more appropriate. While the first algorithm is faster in terms of time complexity (O(n) vs O(n log n)), its space complexity is quadratic ($O(n^2)$), which can lead to rapid memory exhaustion as the input size grows. In contrast, the second algorithm offers a much better space-to-input ratio with linear space complexity (O(n)). In systems where memory is the limiting factor, sacrificing a small amount of computational speed to ensure the program can actually execute within the available RAM is the standard and necessary trade-off.

Save Answer